Week 1 Lab Part 1: Linear Processing and Tracing

Work with your assigned partner.

For each function, write actual Python code on this handout. You may discuss every part of the work with your partner, but **switch who is doing the handwriting when directed below**.

Assume that list arguments are ordinary Python list, containing numbers (ints or floats, it doesn't matter).

When you have completed:

- your `minimum` implementation;
- your `argmin` implementation;
- the instrumentation of `argmin`; and
- your predicted trace of the example above,

bring this handout to me.

You should reach this checkpoint during today's lab period.

---

1. `minimum(lst)`

Write a function:

```
def minimum(lst):
```

that returns the smallest value in `lst`.

You may assume that `lst` is not empty. Do not use Python's built-in `min()` function.

Handwriting

One partner should do the writing for this exercise.

---

2. `argmin(lst)`

Switch writers now. The partner who did *not* write the previous solution should do the handwriting for this exercise.

Write a function:

```
def argmin(lst):
```

that returns the **index** of the smallest value in `lst`.

You may assume that `lst` is not empty. If the smallest value occurs more than once, return the index of its **first** occurrence. Do not use Python's built-in `min()` function.

For example, `argmin([8, 5, 7, 2, 9])` should return:

```
3
```

And `argmin([8, 2, 7, 2, 9])` should return:

```
1
```

Instrument and trace your function

Add one or more `print()` statements to your `argmin` function so that someone reading the output can follow the important state of the algorithm as it works.

You must decide what information needs to be printed, where in the function it should be printed, and what the output means. Do **not** merely print the entire list repeatedly.

Then, **without running the code**, write exactly what your instrumented function will print when called as follows:

```
argmin([8, 5, 7, 2, 9])
```

Write the output on this page, in order as it would appear.

CHECKPOINT

When you get this far, bring it to me to check off. You should reach this checkpoint during today's lab period.

After you are checked off, continue with the longer version in D2L. You should work as a pair during class; after class you may work with whomever you choose.

Continue after the checkpoint

You are expected to be able to do and understand both of the following exercises.

3. `is_sorted(lst)`

Write a function:

```
def is_sorted(lst):
```

that returns `True` if the values in `lst` are in nondecreasing order, and `False` otherwise.

Thus:

```
is_sorted([2, 4, 4, 7, 10])
```

returns `True`, while:

```
is_sorted([2, 4, 7, 6, 10])
```

returns `False`.

Do not use `sorted()`.

Trace it

Decide for yourselves what information would be useful to print while tracing this algorithm.

Instrument your function appropriately.

Then write out the trace for **two examples**:

1. a list that is completely sorted;
2. a list that is mostly sorted, but contains one place where the order is wrong.

Choose your own example lists.

For each example, write the input list and then exactly what your instrumentation would print.

---

4. `has_adjacent_duplicate(lst)`

Write a function:

```
def has_adjacent_duplicate(lst):
```

that returns `True` if two neighboring elements of `lst` are equal.

For example:

`has_adjacent_duplicate([3, 8, 8, 2])`

returns `True`.

`has_adjacent_duplicate([3, 8, 2, 8])`

returns `False`.

Notice that the second list contains a repeated value, but the two copies are **not adjacent**.

Stretch exercises

These are optional. Do them if you have finished the work above and want something harder.

5. `index_closest_to(lst, target)`

Write a function:

```
def index_closest_to(lst, target):
```

that returns the index of the element of `lst` whose value is closest to `target`.

For example:

`index_closest_to([3, 10, 18, 25], 20)`

returns:

`2`

because `18` is closest to `20`.

You may assume that `lst` is not empty.

If two values are equally close, return the index of the first one.

Do not use `min()`.

---

6. `longest_run(lst, value)`

Write a function:

```
def longest_run(lst, value):
```

that returns the length of the longest consecutive run of `value` in `lst`.

For example:

```
longest_run([4, 7, 7, 7, 2, 7, 7], 7)
```

returns:

```
3
```

and:

```
longest_run([4, 2, 9], 7)
```

returns:

```
0
```